Setup

Laboratórios de Sistemas e Serviços

Mário Antunes

Universidade de Aveiro

2026

Introduction

Setting Up Your Digital Workspace

Goal for Today: Ensure everyone has a consistent and powerful work environment. This helps us learn faster and avoids the classic “but it works on my machine!” problem.

We will cover:

- What an Operating System (OS) is and how it is structured.
- How filesystems organize your data on Windows and Linux.
- Why we standardize on Linux for this course.
- Three practical methods to set up a Linux environment.

Operating Systems

What is an Operating System (OS)?

An OS is the foundational software layer that manages all hardware resources and provides services to application programs. Its core responsibilities include:

- **Process Management:** Creating, scheduling, and terminating processes (running programs).
- **Memory Management:** Allocating and deallocating RAM for running processes; implementing virtual memory.
- **Storage Management:** Reading and writing data to disks through filesystem drivers.
- **I/O Management:** Coordinating access to peripherals (keyboard, display, network card, USB devices).
- **Security and Access Control:** Authenticating users and enforcing file permissions.

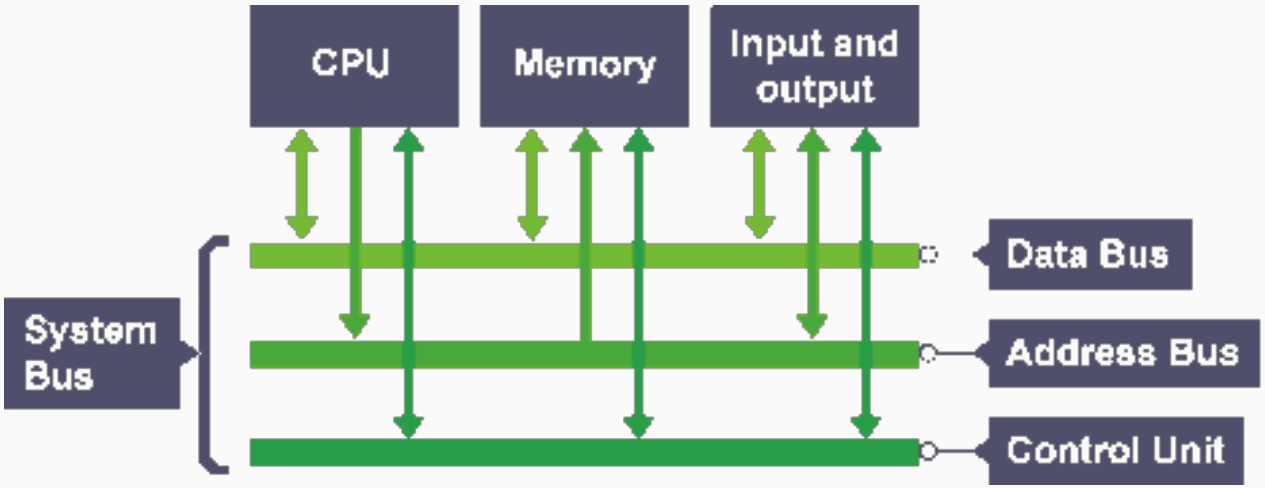
OS Families

We will focus on two main OS families:

- **Windows:** The most common desktop OS, used by the majority of personal computers worldwide.
- **Linux:** A powerful, open-source OS family, dominant in servers, cloud computing, and scientific research.

The Computer: Hardware Overview i

A computer system is built from three fundamental components working together.



The Computer: Hardware Overview ii

- **Processor (CPU):** The Central Processing Unit executes instructions. Modern CPUs have multiple cores, enabling parallel execution of tasks.
- **Memory (RAM):** Random Access Memory provides fast, temporary storage for data and instructions that the CPU is actively using. It is **volatile** (data is lost when power is off).
- **Storage (Disk/SSD):** Provides persistent, long-term storage for files, programs, and the OS itself. It is **non-volatile** but significantly slower than RAM.

The OS acts as the intermediary, deciding which process gets CPU time, how much RAM each process receives, and how data flows to and from storage.

OS Architecture: The Layered Model i

An OS is organized in layers, from hardware at the bottom to user applications at the top:

- **Hardware:** The physical components (CPU, RAM, disk, network interface card).
- **Kernel:** The core of the OS. It runs in privileged mode and has direct access to hardware. It handles process scheduling, memory allocation, device drivers, and filesystem operations.

OS Architecture: The Layered Model ii

- **System Libraries and Services:** Provide a standardized interface (e.g., the C standard library, `libc`) so applications do not need to interact with the kernel directly.
- **Shell and Utilities:** The command-line interface (e.g., Bash) and essential tools (`ls`, `cp`, `grep`) that allow users to interact with the system.
- **User Applications:** Programs like web browsers, text editors, and your own code.

Filesystems

What is a Filesystem?

A filesystem defines how data is organized, stored, and retrieved on a storage device. Without a filesystem, data on a disk would be an undifferentiated stream of bytes with no way to tell where one file ends and another begins.

Windows: NTFS

NTFS (New Technology File System) is the default filesystem on modern Windows installations.

- Uses **drive letters** to identify volumes (e.g., `C:` for the system drive, `D:` for a secondary partition).
- Path separator is a **backslash** (`\`).
- Supports file permissions via **Access Control Lists (ACLs)**.
- Example path:
  `C:\Users\YourName\Documents\report.txt`

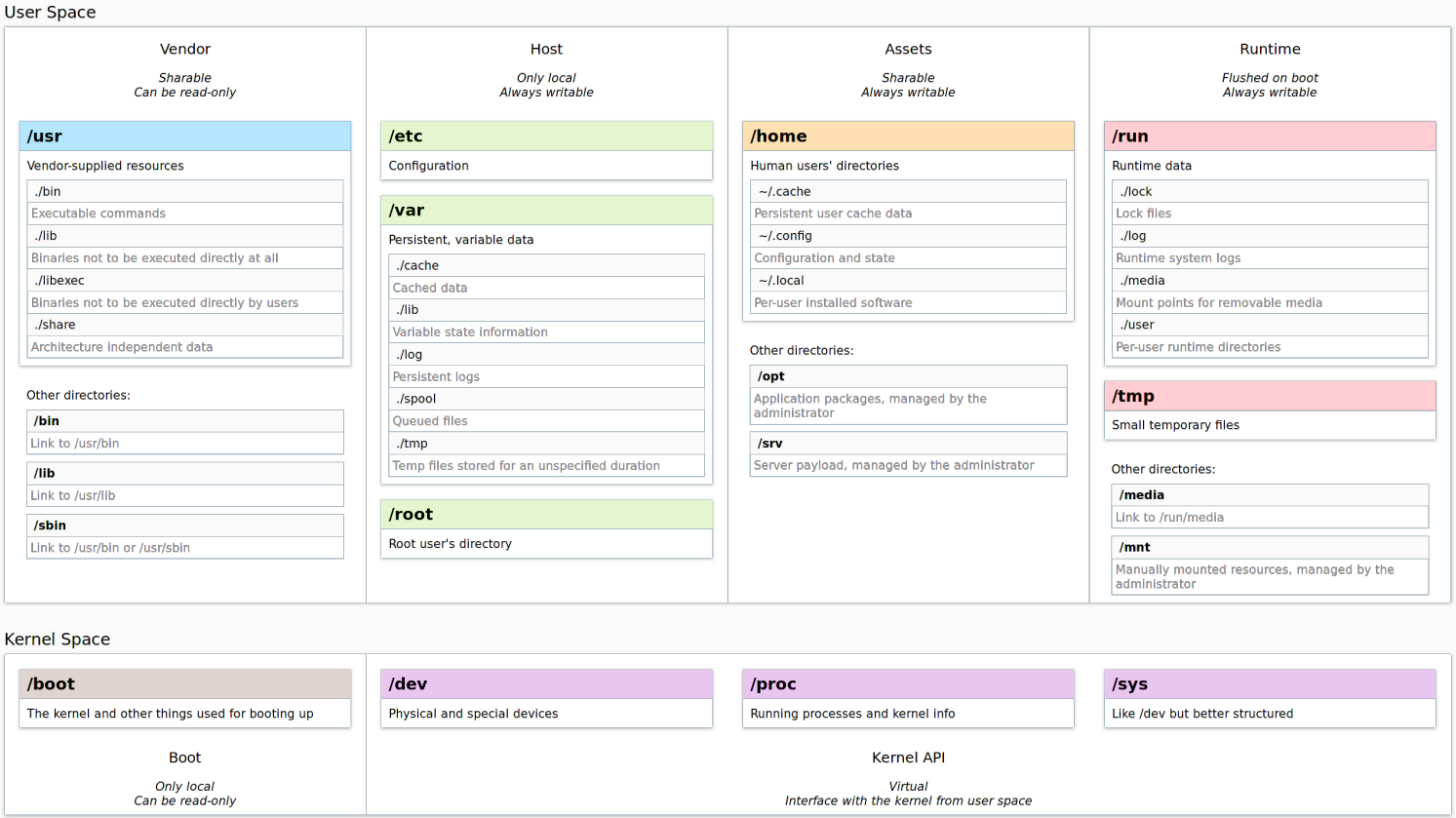
Linux: ext4

ext4 (Fourth Extended Filesystem) is the most common Linux filesystem (others include Btrfs and XFS).

- Has a single, unified **root directory** (`/`). There are no drive letters.
- Everything is represented as a file, including hardware devices (e.g., `/dev/sda1`).
- Path separator is a **forward slash** (`/`).
- Uses a **permission model** based on owner, group, and others (read, write, execute).
- Example path: `/home/yourname/documents/report.txt`

The Linux Filesystem Hierarchy Standard (FHS)

Linux organizes its directories following a well-defined standard. Key directories include:



Sources : man file-hierarchy(7) & man udisks(8).

Key Linux Directories

Directory	Purpose
`/`	Root of the entire filesystem hierarchy
`/home`	Home directories for regular users
`/etc`	System-wide configuration files
`/var`	Variable data: logs, caches, spool files
`/usr`	User programs, libraries, and documentation
`/tmp`	Temporary files (cleared on reboot)
`/dev`	Device files representing hardware
`/proc`	Virtual filesystem exposing kernel and process info

Key takeaway: Understanding path structures is essential for navigating the system, writing scripts, and running programs from the command line.

Why Linux?

Why a Standard Environment? i

Industry Dominance:

- Over **96%** of the world's top 1 million web servers run Linux.
- All major cloud platforms (AWS, Google Cloud, Azure) use Linux as their primary OS.
- Android, the world's most popular mobile OS, is built on the Linux kernel.

Powerful Tooling:

- Offers a rich ecosystem of command-line tools for text processing (`grep`, `sed`, `awk`), automation (`bash`, `cron`), and development (`gcc`, `make`, `git`).
- Package managers (`apt`, `dnf`) make installing software straightforward and reproducible.

Why a Standard Environment? ii

Transparency and Control:

- Open-source: you can inspect, modify, and learn from the source code.
- Encourages understanding of how systems actually work, rather than relying on graphical abstractions.

Reproducibility:

- A shared Linux environment means everyone in the course works with the same tools, same paths, and same behavior. This eliminates configuration-related bugs and makes it easier to help each other.

Now, let us explore your options for getting this environment set up.

Setting Up Linux

Your Three Paths to Linux

1. **Native Linux Installation:**
   - Install Linux directly on your hardware as the primary (or secondary) OS.
   - **Best for:** Maximum performance and full immersion.
2. **Virtual Machine (VM):**
   - Run a complete Linux system inside a window on your current OS, using virtualization software.
   - **Best for:** Safe experimentation with full isolation from your host system.
3. **Windows Subsystem for Linux (WSL):**
   - Run a real Linux kernel and environment directly inside Windows, with deep integration between them.
   - **Best for:** Windows users who want near-native Linux performance without rebooting or running a full VM.

Each option has trade-offs. Let us examine them in detail.

Native Linux Installation

Native Linux Installation i

This means installing a Linux distribution (such as Ubuntu or Fedora) directly on your computer's hardware, either replacing your current OS or alongside it in a **dual-boot** configuration.

How Dual-Boot Works

- The bootloader (typically GRUB) presents a menu at startup, letting you choose between Windows and Linux.
- Each OS occupies its own disk partition and operates independently.
- Only one OS runs at a time, so the running OS has full access to all hardware resources.

Native Linux Installation ii

Pros

- **Best Performance:** No virtualization overhead; Linux has direct access to CPU, GPU, RAM, and all peripherals.
- **Full Immersion:** Forces you to learn the Linux environment thoroughly.
- **Full Hardware Access:** Direct access to GPU acceleration, USB devices, and networking hardware.

Native Linux Installation iii

Cons

- **Complex Setup:** Disk partitioning carries a real risk of data loss if done incorrectly. **Backups are essential.**
- **Hardware Compatibility:** Some hardware (specific Wi-Fi chipsets, fingerprint readers, certain GPUs) may require manual driver installation or kernel configuration.
- **Inconvenient Switching:** Changing between Windows and Linux requires a full reboot.

Who is this for?

Students who are comfortable with computer hardware, enjoy learning by doing, or have a spare machine to dedicate to Linux.

Setup Steps

1. **Choose a distribution:** We recommend **Ubuntu 26.04 LTS** or **Debian 13** for their stability and extensive community support.
2. **Create a bootable USB drive:** Use Rufus (Windows) or BalenaEtcher (cross-platform).
3. **Back up your data.** This step is non-negotiable.
4. **Partition your hard drive** during installation. Allocate at least 40 GB for the Linux partition.
5. **Boot from the USB drive** and follow the installer instructions.

Virtual Machine

Virtual Machine (VM) i

A Virtual Machine uses a **hypervisor** to emulate a complete computer system inside your existing OS. The hypervisor creates an abstraction layer that presents virtual hardware (CPU, RAM, disk, network) to the guest OS.

Types of Hypervisors

- **Type 1 (Bare-metal):** Runs directly on hardware, without a host OS. Examples: VMware ESXi, Microsoft Hyper-V, Xen. Used in data centers and enterprise environments.
- **Type 2 (Hosted):** Runs as an application on top of a host OS. Examples: VirtualBox, VMware Workstation, UTM. This is what we use in this course.

Virtual Machine (VM) ii

Resource Allocation

When creating a VM, you assign it a fixed share of your system's resources:

- **CPU cores:** Typically 2 or more virtual cores.
- **RAM:** At least 2 GB for the VM (your host system should have 8 GB+ total).
- **Disk:** A virtual disk file (typically 20–40 GB) stored on your host filesystem.

Virtual Machine (VM) iii

Pros

- **Safe and Isolated:** The VM is a sandbox. If you break the guest OS, your host is unaffected. You can take **snapshots** to save the VM state and roll back at any time.
- **Easy Setup:** Install VirtualBox, import the provided course image, and start working.
- **Portable:** The VM image (`.ova` file) can be copied to another machine.

Virtual Machine (VM) iv

Cons

- **Resource Heavy:** Running two full operating systems simultaneously demands significant RAM and CPU. Systems with less than 8 GB of RAM will struggle.
- **Slower Performance:** The virtualization layer introduces overhead, especially for disk I/O and graphics.
- **Limited GPU Access:** 3D acceleration and GPU passthrough are limited in Type 2 hypervisors.

Networking Modes

The hypervisor offers several networking modes:

- **NAT (Network Address Translation):** The default mode. The VM shares the host's IP address. Outbound connections work transparently; inbound connections require port forwarding. This is the simplest and most common option.
- **Bridged Adapter:** The VM gets its own IP address on the physical network, as if it were a separate physical machine. Useful when you need the VM to be accessible from other devices on the network.
- **Host-Only:** Creates a private network between the host and the VM only. No internet access, but useful for isolated testing.

Note for Mac Users (Apple Silicon: M1/M2/M3/M4)

VirtualBox has limited support for ARM-based Apple Silicon chips. If you use a Mac with Apple Silicon, we recommend **UTM** (free, open-source) or **VMware Fusion** (free for personal use) instead.

Who is this for?

This is the **recommended default option** for the course. It is the safest, most consistent, and requires no changes to your existing OS.

Setup Steps

1. **Install the Hypervisor:**
   - Windows / Intel Mac: Download and install VirtualBox.
   - Apple Silicon Mac: Download and install UTM.
2. **Download the Course VM Image:** Obtain the `.ova` file from the course website.
3. **Import the Appliance:** In VirtualBox, go to `File > Import Appliance`, select the `.ova` file, and follow the prompts. Adjust RAM and CPU allocation if needed.
4. **Start the VM:** Select the imported machine and click **Start**.
5. **Verify:** Open a terminal inside the VM and run `uname -a` to confirm you are running Linux.

Windows Subsystem for Linux

Windows Subsystem for Linux (WSL)

WSL allows you to run a genuine Linux kernel and userspace directly on Windows, without the overhead of a full virtual machine. Microsoft developed WSL to bring native Linux compatibility to Windows.

WSL 1 vs. WSL 2

Feature	WSL 1	WSL 2
Architecture	Translation layer (syscall mapping)	Lightweight VM with real Linux kernel
Filesystem Performance	Slower on Linux files	Fast on Linux files (`/home`)
System Call Compatibility	Partial	Full
Networking	Shares host network stack	Virtual network adapter
Memory Usage	Lower	Dynamic (grows/shrinks as needed)

We use WSL 2, which ships a real Linux kernel in a lightweight, managed virtual machine. It offers full system call compatibility and excellent performance.

How it Works: Filesystem Integration

- Your Windows drives are automatically mounted inside Linux under `/mnt/`. For example, `C:\Users\YourName` is accessible at `/mnt/c/Users/YourName`.
- The Linux filesystem lives in a separate virtual disk, accessible from Windows Explorer by navigating to `\\wsl$\Ubuntu\home\yourname`.

Important: For best performance, always store your project files inside the Linux filesystem (`/home/yourname/`), not on the mounted Windows drives (`/mnt/c/`). Cross-filesystem access is significantly slower.

How it Works: Networking

- WSL 2 uses a virtual network adapter with its own IP address.
- Internet access works transparently through the host.
- To access a server running inside WSL from Windows, use `localhost` (recent Windows builds support this automatically).

Windows Subsystem for Linux (WSL) i

Pros

- **Excellent Performance:** Near-native speed for command-line tools, compilers, and scripting.
- **GUI Application Support:** WSL 2 supports running Linux graphical applications (via WSLg) alongside Windows apps.
- **Seamless Integration:** Call Linux commands from PowerShell (`wsl ls -la`) and Windows executables from Linux (`explorer.exe .`). Share environment variables and clipboard.
- **Low Resource Usage:** The lightweight VM starts in seconds and uses memory dynamically.

Windows Subsystem for Linux (WSL) ii

Cons

- **Windows Only:** Not available on macOS or native Linux (obviously).
- **Advanced Networking Complexity:** Port forwarding, firewall rules, and USB device access require additional configuration compared to a full VM.
- **Subtle Differences:** Line endings (`\r\n` vs `\n`), file permissions, and path formats can cause issues if you mix Windows and Linux filesystems carelessly.

Who is this for?

Windows users who want a fast, deeply integrated Linux environment without the overhead of a full VM or the commitment of a native installation.

Setup Steps i

1. **Enable WSL:** Open **PowerShell as Administrator** and run:

   ```
   wsl --install
   ```

   This command enables the required Windows features, downloads the Linux kernel, and installs **Debian** by default. To install a different distribution:

   ```
   wsl --install -d Debian
   ```

2. **Reboot** your computer when prompted.
3. **Create a User Account:** After reboot, a terminal window opens automatically. Create your Linux username and password.

Setup Steps ii

4. **Verify the installation:** Run the following commands:

   ```
   cat /etc/os-release
   uname -r
   ```

5. **Launch anytime:** Open “Debian” (or other Linux) from the Start Menu, or type `wsl` in PowerShell.

Wrap Up

Summary: Choosing Your Environment

Your choice depends on your comfort level, hardware, and preferences.

Feature	Native Install	VM	WSL
Performance	Excellent	Moderate	Excellent
Safety/Isolation	Low	High	Moderate
Ease of Setup	Low	High	Moderate
Hardware Access	Full	Limited	Limited
Host OS Impact	High	None	None
Recommended For	Enthusiasts	**Everyone**	Windows Users

The **Virtual Machine** is the default recommendation for this course due to its safety, consistency, and ease of setup.

Next Steps i

Your Task Now:

1. **Choose one** of the three methods (VM is recommended if unsure).
2. Follow the setup instructions to get your Linux environment running.
3. Open a terminal and confirm it works by running:
   `echo "Hello from Linux!"`
   `uname -a`
4. Be ready for our next session with a working environment.

Next Steps ii

Need Help?

- Check the course website for detailed setup guides and the VM image download.
- Ask your professors or teaching assistants during office hours.
- Collaborate with classmates; many setup issues are common and well-documented.

Getting your environment set up is the first important step. Good luck!